

How much sensing is enough? Collective inference for insect-neuron sensor networks

One-page technical abstract, proofread final (2026-09-10). Every number below is recomputed from `netsim/out/summary.csv` by `python -m netsim.verify_abstract`; the CSV is regenerated by `bash netsim/reproduce.sh`.

Problem. Continuous electrophysiological telemetry from insect neurons is a candidate low-cost sensor for chemical, pharmacological and radiological hazards in industrial settings. The single-sensor question, *how early does one neuron flag a perturbation*, has been asked before; the deployment question has not: given real single-unit statistics, how many sensors, in which layout, and with which collective decision rule are needed to raise an alarm before the first loss of function at the source, at a fixed false-alarm rate, and how well can the network locate that source?

Approach and methodology. Single-unit inter-spike-interval statistics were fitted by maximum likelihood (gamma renewal with absolute refractory period) to openly licensed cockroach (Zenodo 10.5281/zenodo.14281, $n = 12$ units) and locust (10.5281/zenodo.14607, $n = 4$) antennal-lobe recordings; these fits are the only empirical input. A neurotoxic front is simulated as a radial wave (1.5 domain units/s) behind which firing rate decays with time constant 8 s; the endpoint is the first collapse at the source (rate < 5 % of baseline for 2 s), which by construction does not depend on sensor placement. Between 4 and 100 sensors are placed on a jittered grid, uniformly at random, or in three clusters, and report 1 s spike counts with log-normal measurement noise (sd 0, 0.15, 0.30). Three single-node statistics (z-score, one-sided CUSUM, Bayesian online changepoint posterior of a depressed regime) are combined by three network rules (max over sensors; mean over 4 nearest neighbours then max; count above a per-detector level). Each pipeline's threshold is set on 40 null runs to a per-run false-alarm probability of 0.05; realised false-alarm rate comes from 20 held-out null runs, and 20 perturbed runs per cell give detection probability, lead time and localisation error (54 cells $\times$ 9 pipelines).

Key findings. (1) Spatial pooling dominates: averaged over all cells, CUSUM with k-NN pooling detects before collapse in 75 % of runs with a mean lead of 6.8 s and a localisation error of 6.8 % of the domain diagonal, against 63 % and 5.4 s for the same statistic with the max rule, at the same realised false-alarm rate (0.06–0.08). (2) Layout matters as much as count (CUSUM + pooling, noise 0.15): on a grid, 16 sensors reach 95 % detection with ~ 7 s lead, random placement needs 36 for the same, and clustered placement reaches 80 % only at 64 sensors and never reaches 100 % in the range tested. (3) Lead time saturates: even 100 grid sensors give 7–13 s (pooled rules, all noise levels) against a theoretical maximum of 26 s, so beyond ~ 36 sensors the marginal sensor buys localisation, not time. (4) The max rule does not scale for bounded statistics: BOCPD+max falls from 80 % detection at 9 sensors to 10 % at 100, because the null maximum over many sensors pushes the calibrated threshold toward the statistic's ceiling; pooled BOCPD stays at 100 %. (5) With 16 grid sensors the best pipeline keeps ≥ 95 % detection at every noise level tested, up to sd 0.30.

Limitations and future work. Ground truth is simulated: this answers a network-design question, not a biological one, and the isotropic front with exponential decay stands in for real toxicodynamics. Rate is the only feature (1 s bins, 3-bin sustain rule); ISI-shape features and the learned detector in `ml/` are unused. Next: validate the single-node model on recordings with a documented pharmacological intervention, add ISI-shape features to the network statistic, and close the loop with the platform's hardware-in-the-loop nodes.

Reproducibility. `bash netsim/reproduce.sh` runs the 21 unit tests and the full sweep (~ 4 min) and writes `netsim/out/RESULTS.md`, the CSVs and the figures; `python -m netsim.verify_abstract` re-derives every number above from `summary.csv`. The simulator is also an MCP server for AI research agents. MIT licence.